CS 4530: Fundamentals of Software Engineering
Module 16: Coding With AI Agents

Adeel Bhutta, Rob Simmons, and Mitch Wand

Khoury College of Computer Sciences

This Is Just the Beginning of Our Conversation

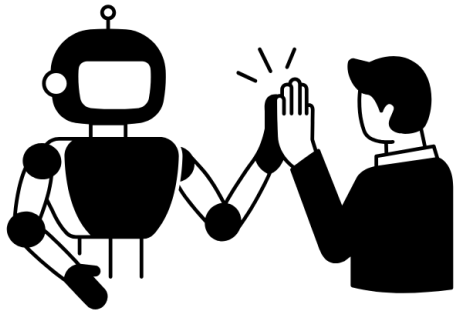
- This topic is unlike anything covered so far:
  - The technology is evolving faster than any textbook (or lecture!) can capture
  - There is genuine disagreement among experts about best practices
  - Your professors are learning alongside you, modeling the best practices we teach

Outline

1. What is an LLM? What is an AI Coding Assistant? Why is context so important?
2. A 4-Step Pattern for Human-AI Workflow
    1. Prompt
    2. Plan
    3. Engage & Evaluate
    4. Finalize & Document
3. A Few Words about AI traps

Our Slogan:

AI amplifies human capabilities; it doesn't replace them

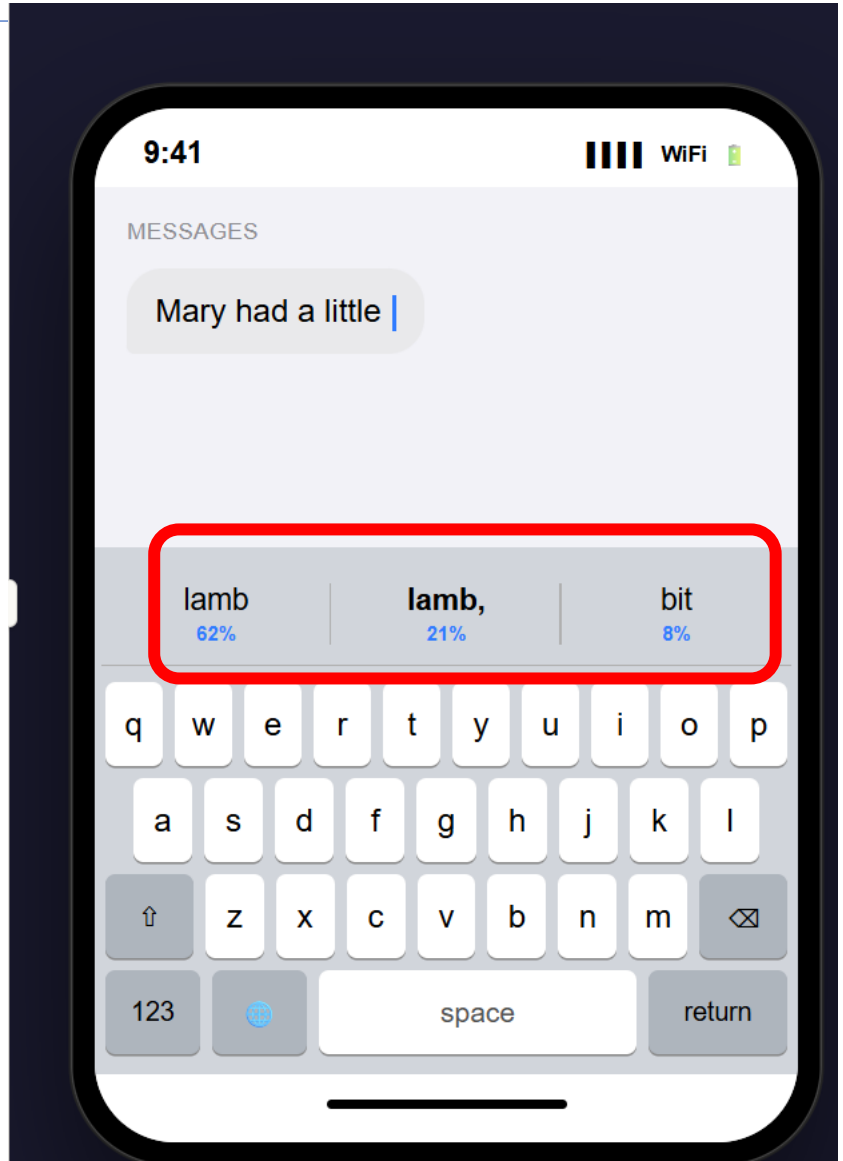


Created by Vector Place
from Noun Project

https://thenounproject.com/browse/icons/term/ai-collaboration (CC BY 3.0)

What is a Large Language Model (LLM?)

- Basically, it is an overgrown autocomplete.
- Given a large database of texts and an initial segment of your input, it answers the question:
- What is the most likely way in which this input would continue?
- More accurately: it produces a probability distribution of likely continuations, and draws from that distribution (hence non-deterministic)



Why this matters

- Explains hallucination: model predicts plausible-looking text, not necessarily true text
- Explains why context matters: predictions depend on what came before
- Explains strengths: patterns in code are highly predictable!

What is an AI Coding Assistant?

- An AI coding assistant typically consists of an LLM
  - trained on a large base of running code
  - operating in a loop with an IDE
    - it can add artifacts inside the IDE to its context
    - it can operate on your code (with your permission)

AI Coding Assistants: Strengths

- **Strengths:**
  - Pattern recognition: recognizes and reproduces common coding patterns across millions of codebases
  - Syntax knowledge: extensive knowledge of languages, libraries, and frameworks
  - Integration with IDE can guide search
  - Cross-domain transfer: can apply patterns from one language to another
  - Rapid prototyping: generates boilerplate, tests, and common implementations quickly
  - AI in-the-loop: can run tests and evaluate the results
    - did it work? if not it can try something else

AI Coding Assistants: Limitations

- **Limitations:**
  - Limited Context: can only see 100K-1M tokens at once — may miss parts of large codebases
  - Generates code based on patterns, not execution results
  - Hallucination: may generate plausible-looking code that doesn’t actually work, or doesn't satisfy your project constraints.
  - Think of it as a well-read junior developer who has seen millions of codebases — but has never run your specific project.

But An LLM Can't Find Everything

- What tools can auto-find: related files, type definitions, call graphs, similar patterns, test files
- What tools cannot auto-find: why a design was chosen, undocumented requirements, knowledge in your head
- And of course, the LLM can't figure out your intent.

A 4-Step Workflow for Human-AI Interaction

1. Prompt
   - Give the AI the context it needs to do its job
2. Plan
   - Have the AI create a plan for what you want it to do
   - Review the plan
   - Review the review
3. Engage & Evaluate
   - Engage with AI to guide to the desired result
   - Evaluate the results
4. Finalize & Document
   - Make a record of design decisions to benefit future teams.

A 4-Step Workflow for Human-AI Interaction

1. Prompt
   - Give the AI the context it needs to do its job
2. Plan
   - Have the AI create a plan for what you want it to do
   - Review the plan
   - Review the review
3. Engage & Evaluate
   - Engage with AI to guide to the desired result
   - Evaluate the results
4. Finalize & Document
   - Make a record of design decisions to benefit future teams.

Prompt: Give the AI the context it needs to do its job

- What are the domain concepts?
- What is the goal?
- What kind of output do we want?
- What restrictions to we need to place on the AI?

Prompts can have different scopes

- Project-level prompts
  - for use throughout the project
  - project constraints (guardrails)
  - project architecture
- Task-level prompts
  - describe specific pieces of the project
  - describe constraints & architecture specific to this task

Project Constraint Prompts

```
## Software Stack
- Frontend: Next.js 14+ App Router, TypeScript strict
- Backend: Convex for real-time data, Supabase for auth + storage
- Auth: Clerk (never roll custom auth, we are not animals)
- Styling: Tailwind only — no CSS modules, no styled-components
## Hard Rules
- Never install a new dependency without asking first
- Never modify the database schema without showing the migration plan
- All API calls go through Convex functions, never direct Supabase
  client calls from components
- Environment variables go in .env.local, never hardcoded
  (I will find you and I will revert you)
# FAILURE CONDITIONS:
- Any component exceeds 150 lines
- Fetches data client-side when it could be server-side
- Uses any UI library besides Tailwind utility classes
- Missing loading and error states
- Missing TypeScript types on any function parameter
```

These are your guardrails

Architecture Constraint Prompts

```
## Repository Structure
- `src/` for source code
  - `additionService.ts` for business logic
  - `additionController.ts` for handling requests
  - `*.test.ts` for tests
  - `scratchpad.ts` for experimental code
  - `express.ts` for express app setup
  - - `src/server.ts` to start the application
`package.json` for dependencies and scripts

## Patterns
- Use server components by default, client components only when
  interactivity is required
- Error boundaries on every route segment
- Zod validation on every user input
```

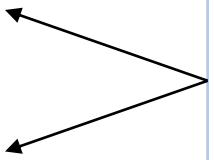
- Autogenerated README.md files can be a good basis for this portion of the prompt

Task Prompts

- These are the cues for specific tasks within your project
- Carefully craft these in advance
- Most AIs will let you save these for reuse
- You can invoke them in chat or in scripts

Task Prompts should have testable Success and Failure Criteria

- Write prompts like like legal contracts, with four enforceable sections:
- **Goal** — a list of testable criteria (COS?!)
- **Output Format** — the specific structure you expect.
- **Constraints** — hard boundaries that can't be crossed.
- **Failure Conditions** — what makes the output unacceptable.



include these if they are different from project-wide criteria

Task Prompt: Goal

```
# Goal (Bad) Add a subscription system to the app
- Add a subscription system to the app

# Goal (Better) Implement Stripe subscription management
## Success
- a user can subscribe to one of 3 tiers (free/pro/team)
- a user can upgrade/downgrade instantly
- a user can see billing status on /settings/billings
- a free user can subscribe to Pro,
- see the charge on Stripe dashboard
- access gated features within 5 seconds.
```

The difference is testability -- each of these can be independently tested.

Task Prompt: Format

What you want delivered, and how you want it delivered

- Tell it what to hand you
- Bad: `Create an API endpoint for user onboarding`
- Better:

```
# FORMAT:
1. Convex function in convex/users.ts (mutation, not action)
2. Zod schema for input validation in convex/schemas/onboarding.ts
3. TypeScript types exported from convex/types/user.ts
4. Include JSDoc on the public function
5. Return { success: boolean, userId: string, error?: string }
```

How much prompting you need to provide depends on three things:

1. **Your understanding:** Experts know exactly what context matters; novices may not know what to include. You can only identify relevant context for things you understand.
2. **Model capabilities:** Frontier models infer much more from less. GPT-3.5 needed explicit instructions; Claude Opus can often infer intent.
3. **Tool capabilities:** Basic Copilot sees only your open files. Cursor and Claude Code index your entire codebase and find files automatically.

- When all three are HIGH: just describe intent — the system figures out context. When any is LOW: you must compensate manually

Prompting Myths That Don’t Actually Help

- Myth: Persona prompts improve output
  - “You are a brilliant 10x engineer...” — Models don’t roleplay
- Myth: Politeness affects performance
  - “Please” and “Thank you” — Won’t affect the model, but polite communication is a good habit for talking to humans!
  - better code. Context and specificity matter, not flattery.
- What actually helps:
  - Relevant Context
  - Specific Requirements
  - Concrete examples
  - Clear success criteria

A 4-Step Workflow for Human-AI Interaction

1. Prompt
   - Give the AI the context it needs to do its job
2. Plan
   - Have the AI create a plan for what you want it to do
   - Review the plan
   - Review the review
3. Engage & Evaluate
   - Engage with AI to guide it to the desired result
   - Evaluate the results
4. Finalize & Document
   - Make a record of design decisions to benefit future teams.

Step 2: Generate a plan first.

- Have the AI generate a plan first; generate code only after plan passes review.
- A plan has a small evaluation surface (easier to review)
- Expert says: “Wait, we should use sessions not JWT for this use case”
  - a plan exposes architectural issues early.
- Learner says: “I don’t know what JWT is — I should learn that before approving!”
  - a plan reveals knowledge gaps.
- Cost to fix at this stage: LOW
- If you can’t evaluate the plan, you can’t evaluate the code. Plans reveal knowledge gaps early.
- Most AIs have a "plan mode" to facilitate this

The plan is your blueprint

- Eventually, you will want to check that the generated code matches the blueprint
- So you will need to know how to evaluate the blueprint!

If you can't evaluate the AI output, maybe you shouldn't be using so much AI (Just sayin')

- **Easy to evaluate — use AI freely:**
  - Does the script run? Does it compile? Does the test pass? (binary yes/no — verify immediately)
- **Hard to evaluate — use AI carefully:**
  - Does the quiz have confusing questions? → Give it to 100 students and find out
  - Is this architecture scalable? → Wait 6 months in production
  - Will users find this intuitive? → Ship and measure
- AI can help you GENERATE for any task. But for hard-to-evaluate outcomes, don’t rely on AI to EVALUATE — you need external validation: user testing, expert review, time in production.

When should I use AI?
Task familiarity determines appropriateness

- **Use AI when**: You have sufficient domain knowledge to evaluate outputs
- **Avoid AI when**: You lack the expertise to assess correctness and quality
- **Learning consideration**: Using AI without foundational knowledge can lead to deskilling

A 4-Step Workflow for Human-AI Interaction

1. Prompt
   - Give the AI the context it needs to do its job
2. Plan
   - Have the AI create a plan for what you want it to do
   - Review the plan
   - Review the review
3. Engage & Evaluate
   - Engage with AI to guide it to the desired result
   - Evaluate the results
4. Finalize & Document
   - Make a record of design decisions to benefit future teams.

Step 3: Engage & Evaluate

- **Calibrate**: Steer AI toward desired outcomes through feedback
  - Example: “That’s close, but use interfaces instead of abstract classes”
  - This is a conversation — you may calibrate 3–4 times per task. This is normal.
- **Tweak**: Manually refine AI-generated artifacts
  - Naming, edge cases, comments, style adjustments. AI output is rarely perfect — this is expected.

Calibration and Tweaking

- Especially applicable for non-code AI-generated artifacts
  - text
  - diagrams
  - slides
  - etc…

Another kind of engagement: Making sure the AI obeys the constraints

- Run the AI in a sandbox, if you can
  - to prevent it from deleting your emails, etc.
- Remind it often about the constraints, so the constraints are always in context
- Manual checking (at each step!)
- Automated review by a separate AI
- Refer to the onboarding guide of your AI for more specific advice.

Step 3, cont'd: Evaluate

- Review code with the plan as a checklist.
- “I know what to look for because I already approved the approach.”
- Remember: If you didn't evaluate the plan, you can’t evaluate the code.
- Advanced usage: some of this review can be automated (with a separate, isolated AI)
- But then you still have to review the review!

You probably need a plan to do the evaluation

- Assess AI outputs against expected results utilizing domain knowledge.
- Compare output against expected end results and other success criteria.

A 4-Step Workflow for Human-AI Interaction

1. Prompt
   - Give the AI the context it needs to do its job
2. Plan
   - Have the AI create a plan for what you want it to do
   - Review the plan
   - Review the review
3. Engage & Evaluate
   - Engage with AI to guide it to the desired result
   - Evaluate the results
4. Finalize & Document
   - Make a record of design decisions to benefit future teams.

Finalizing

- Before committing AI-generated code, ask yourself:
  - Can I justify the design decisions in this code to a colleague?
  - If I look at this in 6 months, will I know WHY I chose this approach?
  - Am I prepared to take responsibility if this code is wrong?
  - Would a new team member understand the design decisions?
- If the answer to any of these is “no” — you are creating maintenance debt that future-you will pay.
- “It worked” is not a reason. “The AI suggested it” is not a reason. Document the actual design rationale.

Use Design As a Way of Communicating Organization

Remember this slide from Module 08 Code Level Design?

- Software systems must be comprehensible by humans
- Which humans?
  - The other members of your team
  - The folks who will maintain and modify your system
  - Management
  - Your clients
  - and ...
  - You, a week from now or 6 weeks from now

The future of H/AI workflows: “Teams of agents”??

- So far in 2026, there have been some unhinged, heavily hyped versions of having different agents with different instructions working together. (Search for “Gas Town Yegge” at your own risk)
- One of Prof Simmons’s colleagues at Lean FRO, Kim Morrison, recently demoed a less unhinged version of this idea.
- Nobody at present thinks this kind of workflow produces good software engineering — software maintainable *over time* — but it is demonstrably better at doing certain goal-oriented complex tasks (like searching for bugs in a compiler).

4. A few words about AI traps

Trap #1: The “Vibe Coding” Trap

- What is “vibe coding”?
  - Evaluating only execution, not code. Ask AI to implement a feature. Run the app, see if it “works.” If error, describe the error to AI and repeat. Never actually read or understand the code.
- Why it leads to collapse:
  - No troubleshooting capability — you don’t understand what the code does
  - Can’t provide effective feedback — you can only describe symptoms, not problems
  - Brittle development — one change breaks everything and you don’t know why
- To effectively use AI, you must be able to evaluate the code itself — not just “does it run?”

Trap #2: AI Creates “Learning Debt” When Used Too Early

- **Path A — Learning First (recommended):**
  - Manual coding, mistakes, understanding errors, building mental models. Slower at first.
  - When you eventually use AI, you can evaluate and guide it effectively.
  - Solid foundation that supports long-term productivity growth.
- **Path B — AI First:**
  - Fast initial progress. But you never learn WHY the code works.
  - When complex bugs hit, you’re stuck — you can’t even describe the problem to AI effectively.
  - Productivity collapses and never recovers to Path A levels.

A "learning-tax" strategy can help mitigate deskilling.

- Adopt a “learning tax” strategy: deliberately choose to implement certain components manually, even when AI could generate them instantly.
- Otherwise you won't recognize when the AI is screwing up!
- The goal isn’t to code without AI or to use it for everything, but to maintain the expertise that makes you irreplaceable—the judgment, creativity, and deep understanding that transforms good code into great software. (--Jon Bell)

Review: Our outline

1. What is an LLM? What is an AI Coding Assistant? Why is context so important?
2. A 4-Step Pattern for Human-AI Workflow
    1. Prompt
    2. Plan
    3. Engage & Evaluate
    4. Finalize & Document
3. A Few Words about AI traps